

Plan de Migración de Datos

Firestore (NoSQL) → PostgreSQL (Relacional)

Proyecto KaiOra Institute — Duoc UC

1. Contexto y Justificación de la Migración

KaiOra fue desarrollado inicialmente sobre Firestore, una base de datos NoSQL orientada a documentos. Esta elección fue técnicamente justificada por el patrón de acceso dominante de la aplicación: la carga completa de una ficha técnica en una sola operación de lectura, aprovechando el modelo desnormalizado para evitar JOINS costosos en lecturas repetitivas.

Sin embargo, a medida que el sistema escala hacia nuevos requerimientos —reportes históricos de evaluaciones, auditoría de activaciones por curso, análisis estadístico del desempeño por alumno— el modelo documental presenta limitaciones estructurales:

- **Consultas analíticas complejas** requieren múltiples lecturas secuenciales en Firestore, mientras que en PostgreSQL se resuelven con una sola consulta SQL con JOINS y agregaciones.
- **Integridad referencial** no está garantizada en Firestore a nivel de motor: un documento `course-recipes` puede referenciar un `recipeId` que ya no existe. PostgreSQL resuelve esto con `FOREIGN KEY` y `ON DELETE CASCADE`.
- **Transacciones ACID**: Firestore soporta transacciones, pero con limitaciones de escritura (máximo 500 documentos por transacción). PostgreSQL ofrece transacciones ACID completas sin restricciones de volumen.
- **Costo a escala**: El modelo de facturación de Firestore por operación de lectura/escritura se vuelve costoso a medida que crecen los reportes y las consultas analíticas.

La migración a PostgreSQL representa una evolución arquitectónica que preserva la integridad histórica de los datos mientras habilita las capacidades relacionales necesarias para la siguiente fase del producto.

2. Mapeo de Modelos: Firestore → PostgreSQL

2.1 Colecciones Firestore actuales

Colección Firestore	Naturaleza	Complejidad de migración
<code>users</code>	Documento plano	Baja
<code>courses</code>	Documento con array <code>studentIds[]</code>	Media
<code>technical-sheets</code>	Documento profundamente anidado	Alta
<code>course-recipes</code>	Documento plano con array <code>observations[]</code>	Media

2.2 Transformación de arrays anidados

El mayor desafío de la migración es la **desnormalización inversa**: los arrays embebidos en los documentos de `technical-sheets` deben expandirse a filas individuales en tablas relacionales.

Documento Firestore (técnical-sheet):

```
{
  "id": "abc123",
  "name": "Risotto de Espárragos",
  "profesorId": "uid_profesor",
  "ingredients": [
    { "name": "Arroz Arborio", "quantity": 320, "unit": "gr" },
    { "name": "Caldo de Verduras", "quantity": 800, "unit": "ml" }
  ],
  "procedimiento": [
    { "orden": 1, "descripcion": "Sofreír la cebolla en mantequilla." },
    { "orden": 2, "descripcion": "Agregar el arroz y nacarar 2 minutos." }
  ],
  "pcc": ["Mantener temperatura > 75°C en el centro del alimento"],
  "keyPoints": ["Incorporar el caldo caliente de a poco"],
  "erroresFrecuentes": ["Agregar todo el caldo de una vez"],
  "evaluaciones": ["Textura del risotto", "Presentación del plato"]
}
```

Modelo relacional equivalente (PostgreSQL):

```
recetas (1) —< ingredientes (N)
recetas (1) —< pasos_procedimiento (N)
recetas (1) —< puntos_criticos (N)
recetas (1) —< puntos_clave (N)
recetas (1) —< errores_frecuentes (N)
recetas (1) —< criterios_evaluacion (N)
```

Nota de diseño: Se optó por tablas independientes para cada array en lugar del tipo nativo `TEXT[]` de PostgreSQL. Aunque `TEXT[]` reduciría el número de tablas, sacrifica la capacidad de indexar, filtrar y agregar sobre elementos individuales de la lista. Por ejemplo, una consulta como *"¿cuántas recetas comparten el mismo punto crítico?"* es trivial con una tabla `puntos_criticos` pero imposible con `TEXT[]`.

3. Esquema DDL — PostgreSQL

```
--  
-- TABLAS MAESTRAS  
--  
  
CREATE TYPE rol_usuario AS ENUM ('admin', 'profesor', 'alumno');  
CREATE TYPE estado_ficha AS ENUM ('Activa', 'Archivada');  
CREATE TYPE dificultad_receta AS ENUM ('Fácil', 'Media', 'Difícil');  
  
CREATE TABLE usuarios (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  firebase_uid VARCHAR(128) UNIQUE NOT NULL, -- preservar para migración sin  
ruptura de Auth  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  nombre      VARCHAR(255) NOT NULL,  
  rol         rol_usuario NOT NULL,  
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE cursos (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nombre      VARCHAR(255) NOT NULL,  
  descripcion TEXT,  
  imagen_url  TEXT,  
  profesor_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,  
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
--  
-- TABLAS DE NEGOCIO: RECETAS  
--  
  
CREATE TABLE recetas (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  firebase_id  VARCHAR(128) UNIQUE, -- ID original de Firestore para trazabilidad
```

```

nombre      VARCHAR(255) NOT NULL,
descripcion  TEXT NOT NULL,
categoria    VARCHAR(100) NOT NULL,
dificultad  dificultad_receta NOT NULL,
rendimiento  INT NOT NULL CHECK (rendimiento > 0),
tiempo_prep_min INT NOT NULL CHECK (tiempo_prep_min >= 0),
imagen_url   TEXT,
profesor_id  UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,
estado       estado_ficha NOT NULL DEFAULT 'Archivada',
creado_en    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
actualizado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE ingredientes (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  receta_id UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,
  nombre  VARCHAR(255) NOT NULL,
  cantidad NUMERIC(10, 2) NOT NULL CHECK (cantidad > 0),
  unidad  VARCHAR(20) NOT NULL,
  orden   INT NOT NULL DEFAULT 0
);

```

```

CREATE TABLE pasos_procedimiento (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  receta_id UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,
  orden   INT NOT NULL,
  descripcion TEXT NOT NULL,
  UNIQUE (receta_id, orden) -- no puede haber dos pasos con el mismo número en la
misma receta
);

```

```

CREATE TABLE puntos_criticos (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  receta_id UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,
  descripcion TEXT NOT NULL,
  orden   INT NOT NULL DEFAULT 0
);

```

```

CREATE TABLE puntos_clave (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  receta_id UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,
  descripcion TEXT NOT NULL,
  orden   INT NOT NULL DEFAULT 0
);

```

```

CREATE TABLE errores_frecuentes (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  receta_id UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,

```

```
    descripcion TEXT NOT NULL,  
    orden      INT NOT NULL DEFAULT 0  
);
```

```
CREATE TABLE criterios_evaluacion (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    receta_id   UUID NOT NULL REFERENCES recetas(id) ON DELETE CASCADE,  
    descripcion TEXT NOT NULL,  
    orden      INT NOT NULL DEFAULT 0  
);
```

```
-- _____  
-- TABLAS DE ASOCIACIÓN  
-- _____
```

```
CREATE TABLE inscripciones (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    alumno_id   UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,  
    curso_id    UUID NOT NULL REFERENCES cursos(id) ON DELETE CASCADE,  
    inscrito_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    UNIQUE (alumno_id, curso_id) -- un alumno no puede estar dos veces en el mismo  
curso  
);
```

```
CREATE TABLE fichas_activas (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    receta_id   UUID NOT NULL REFERENCES recetas(id) ON DELETE RESTRICT,  
    curso_id    UUID NOT NULL REFERENCES cursos(id) ON DELETE CASCADE,  
    profesor_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,  
    activado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    UNIQUE (receta_id, curso_id) -- una receta solo puede estar activa una vez por curso  
);
```

```
CREATE TABLE observaciones_ficha (  
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    ficha_activa_id UUID NOT NULL REFERENCES fichas_activas(id) ON DELETE  
CASCADE,  
    descripcion TEXT NOT NULL,  
    orden      INT NOT NULL DEFAULT 0  
);
```

```
-- _____  
-- ÍNDICES DE RENDIMIENTO  
-- _____
```

```
CREATE INDEX idx_recetas_profesor ON recetas(profesor_id);  
CREATE INDEX idx_recetas_estado ON recetas(estado);  
CREATE INDEX idx_ingredientes_receta ON ingredientes(receta_id);
```

```
CREATE INDEX idx_pasos_receta ON pasos_procedimiento(receta_id, orden);
CREATE INDEX idx_fichas_activas_curso ON fichas_activas(curso_id);
CREATE INDEX idx_inscripciones_alumno ON inscripciones(alumno_id);
```

4. Estrategia de Migración ETL

Fase 1 — Extracción (Firebase → JSON)

Se utilizará el SDK de administración de Firebase (**firebase-admin**) para exportar las colecciones a archivos JSON estructurados. Este proceso no afecta la operación del sistema ya que es de solo lectura.

```
// export-firestore.js
const admin = require('firebase-admin');
const fs = require('fs');

admin.initializeApp({ credential: admin.credential.applicationDefault() });
const db = admin.firestore();

const collections = ['users', 'courses', 'technical-sheets', 'course-recipes'];

async function exportCollection(name) {
  const snapshot = await db.collection(name).get();
  const data = snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
  fs.writeFileSync(`./export/${name}.json`, JSON.stringify(data, null, 2));
  console.log(`✓ ${name}: ${data.length} documentos exportados`);
}

(async () => {
  for (const col of collections) await exportCollection(col);
})();
```

Script de prevalidación — ejecutar antes de continuar:

```
// validate-export.js
const users = require('./export/users.json');
const courses = require('./export/courses.json');
const sheets = require('./export/technical-sheets.json');
const relations = require('./export/course-recipes.json');

let errors = 0;

// Validar emails únicos y no nulos
const emails = users.map(u => u.email);
```

```

const duplicateEmails = emails.filter((e, i) => emails.indexOf(e) !== i);
if (duplicateEmails.length) {
  console.error(` ✗ Emails duplicados: ${duplicateEmails.join(', ')} `);
  errors++;
}

// Validar integridad de course-recipes
const sheetIds = new Set(sheets.map(s => s.id));
const courseIds = new Set(courses.map(c => c.id));
const userIds = new Set(users.map(u => u.id));

relations.forEach(r => {
  if (!sheetIds.has(r.recipeId)) { console.error(` ✗ Huérfano: recipeId ${r.recipeId} no existe`);
  errors++; }
  if (!courseIds.has(r.courseId)) { console.error(` ✗ Huérfano: courseId ${r.courseId} no
  existe`); errors++; }
  if (!userIds.has(r.profesorId)) { console.error(` ✗ Huérfano: profesorId ${r.profesorId} no
  existe`); errors++; }
});

// Validar campos críticos de recetas
sheets.forEach(s => {
  if (!s.name) { console.error(` ✗ Receta sin nombre: ${s.id}`); errors++; }
  if (!s.profesorId) { console.error(` ✗ Receta sin profesor: ${s.id}`); errors++; }
  if (!Array.isArray(s.ingredients) || s.ingredients.length === 0) {
    console.warn(` ⚠ Receta sin ingredientes: ${s.name}`);
  }
});

console.log(errors === 0 ? '✓ Validación exitosa' : ` ✗ ${errors} errores encontrados.
  Corregir antes de continuar.`);

```

Fase 2 — Transformación (Desnormalización inversa)

Esta es la fase más compleja. Los arrays embebidos en cada documento de **technical-sheets** deben **expandirse a filas individuales** para el modelo relacional.

Sub-fase 2.1 — Mapeo de IDs

Los documentos de Firestore usan IDs de tipo **string** auto-generados. En PostgreSQL se usarán **UUID**. Se construye un mapa de correspondencia para mantener la trazabilidad durante la carga:

```

// El campo firebase_id en las tablas recetas y usuarios
// permite referenciar el documento original en caso de discrepancias post-migración
const idMap = {

```

```
users: {}, // firebaseUid → postgresUUID
courses: {}, // firebaseCourseId → postgresUUID
sheets: {}, // firebaseSheetId → postgresUUID
};
```

Sub-fase 2.2 — Expansión de arrays

Para cada documento `technical-sheets`, los arrays se transforman en colecciones de objetos con referencia a su `receta_id`:

```
// transform.js (extracto)
function transformSheet(sheet, recetaUUID) {
  return {
    ingredientes: (sheet.ingredients || []).map((ing, i) => ({
      receta_id: recetaUUID,
      nombre: ing.name,
      cantidad: ing.quantity,
      unidad: ing.unit,
      orden: i
    })),
    pasos: (sheet.procedimiento || []).map(p => ({
      receta_id: recetaUUID,
      orden: p.orden,
      descripcion: p.descripcion
    })),
    pcc: (sheet.pcc || []).map((item, i) => ({
      receta_id: recetaUUID,
      descripcion: item,
      orden: i
    })),
    puntos_clave: (sheet.keyPoints || []).map((item, i) => ({
      receta_id: recetaUUID,
      descripcion: item,
      orden: i
    })),
    errores: (sheet.erroresFrecuentes || []).map((item, i) => ({
      receta_id: recetaUUID,
      descripcion: item,
      orden: i
    })),
    evaluaciones: (sheet.evaluaciones || []).map((item, i) => ({
      receta_id: recetaUUID,
      descripcion: item,
      orden: i
    })),
  };
}
```

Fase 3 — Carga (Inserción secuencial por dependencia)

La carga respeta estrictamente el orden de dependencia de llaves foráneas. Cualquier inserción fuera de orden producirá un error de integridad referencial.

Orden de inserción

1. usuarios ← no depende de nadie
2. cursos ← depende de usuarios (profesor_id)
3. inscripciones ← depende de usuarios y cursos
4. recetas ← depende de usuarios (profesor_id)
5. ingredientes ← depende de recetas
6. pasos_procedimiento ← depende de recetas
7. puntos_criticos ← depende de recetas
8. puntos_clave ← depende de recetas
9. errores_frecuentes ← depende de recetas
10. criterios_evaluacion ← depende de recetas
11. fichas_activas ← depende de recetas y cursos
12. observaciones_ficha ← depende de fichas_activas

Cada bloque de inserción se envuelve en una **transacción atómica**. Si un INSERT falla, se hace rollback del bloque completo:

```
BEGIN;
```

```
INSERT INTO recetas (id, firebase_id, nombre, descripcion, categoria, ...)
VALUES ($1, $2, $3, $4, $5, ...);
```

```
INSERT INTO ingredientes (receta_id, nombre, cantidad, unidad, orden)
VALUES ($1, $2, $3, $4, $5),
      ($1, $6, $7, $8, $9);
```

```
-- ... resto de tablas dependientes de esta receta
```

```
COMMIT;
```

```
-- Si cualquier INSERT falla, el ROLLBACK es automático
```

Fase 4 — Validación y Corte (Ventana de Mantenimiento)

Ventana de mantenimiento: 2 horas

Hora	Acción
------	--------

T+0:00	App pasa a modo "Solo Lectura". Firebase rules: <code>allow read; allow write: if false;</code>
T+0:05	Exportación final incremental de Firestore (capturar cambios desde Fase 1)
T+0:20	Carga incremental en PostgreSQL de los documentos nuevos desde la Fase 1
T+1:00	Ejecución de consultas de validación cruzada
T+1:45	Si validación exitosa: actualizar variables de entorno → apuntar a PostgreSQL
T+2:00	App vuelve a modo lectura/escritura sobre PostgreSQL. Monitoreo activo 1 hora

Consultas de validación cruzada

-- Verificar conteo total de registros

```
SELECT
  (SELECT COUNT(*) FROM usuarios)      AS total_usuarios,
  (SELECT COUNT(*) FROM cursos)        AS total_cursos,
  (SELECT COUNT(*) FROM recetas)       AS total_recetas,
  (SELECT COUNT(*) FROM ingredientes)   AS total_ingredientes,
  (SELECT COUNT(*) FROM fichas_activas) AS total_fichas_activas,
  (SELECT COUNT(*) FROM inscripciones)  AS total_inscripciones;
```

-- Verificar que ninguna receta quedó sin ingredientes (excepto las que no tenían)

```
SELECT r.nombre, COUNT(i.id) AS total_ingredientes
FROM recetas r
LEFT JOIN ingredientes i ON i.receta_id = r.id
GROUP BY r.id, r.nombre
HAVING COUNT(i.id) = 0;
```

-- Verificar integridad de fichas activas

```
SELECT fa.*
FROM fichas_activas fa
LEFT JOIN recetas r ON r.id = fa.receta_id
LEFT JOIN cursos c ON c.id = fa.curso_id
WHERE r.id IS NULL OR c.id IS NULL;
```

-- Resultado esperado: 0 filas

Los conteos de PostgreSQL deben coincidir con los conteos de los archivos JSON exportados en la Fase 1. Una discrepancia de **un solo registro** detiene el corte.

5. Plan de Reversión (Rollback)

El plan de rollback garantiza **cero pérdida de datos** y un tiempo de recuperación menor a 5 minutos.

Condiciones que activan el rollback:

- Cualquier error irrecuperable durante la Fase 3 (fallo de transacción)
- Discrepancia en los conteos de validación de la Fase 4
- Tiempo de ventana de mantenimiento excedido

Procedimiento:

1. Detener inmediatamente el proceso ETL
2. Revertir las variables de entorno de la aplicación a los valores de Firebase
3. En Firebase Console: desactivar el modo "Solo Lectura" (restaurar reglas originales)
4. Verificar que la app responde correctamente sobre Firebase
5. Documentar el error, analizar causa raíz y reprogramar la ventana de migración

Como Firebase se mantuvo en modo **Solo Lectura** (nunca en modo escritura ni fue modificada), los datos están íntegros y disponibles inmediatamente. No se requiere restaurar ningún backup.

6. Impacto en la Capa de Aplicación post-migración

Una vez completada la migración, la capa de servicios de la aplicación debe actualizarse para reemplazar las llamadas al SDK de Firestore por consultas SQL. El impacto principal recae en `firebase.service.ts`, que deberá ser reemplazado por un servicio HTTP que consuma una API REST o GraphQL conectada a PostgreSQL.

La consulta equivalente a `getDocument('technical-sheets/{id}')` en el nuevo modelo:

```
SELECT
  r.*,
  JSON_AGG(DISTINCT JSONB_BUILD_OBJECT('nombre', i.nombre, 'cantidad', i.cantidad,
    'unidad', i.unidad) ORDER BY i.orden) AS ingredientes,
  JSON_AGG(DISTINCT JSONB_BUILD_OBJECT('orden', p.orden, 'descripcion',
    p.descripcion) ORDER BY p.orden) AS procedimiento,
  JSON_AGG(DISTINCT pc.descripcion ORDER BY pc.orden) FILTER (WHERE pc.id IS
    NOT NULL) AS pcc,
  JSON_AGG(DISTINCT pk.descripcion ORDER BY pk.orden) FILTER (WHERE pk.id IS
    NOT NULL) AS puntos_clave,
  JSON_AGG(DISTINCT ef.descripcion ORDER BY ef.orden) FILTER (WHERE ef.id IS NOT
    NULL) AS errores_frecuentes,
  JSON_AGG(DISTINCT ce.descripcion ORDER BY ce.orden) FILTER (WHERE ce.id IS
    NOT NULL) AS criterios_evaluacion
```

```
FROM recetas r
LEFT JOIN ingredientes      i  ON i.receta_id = r.id
LEFT JOIN pasos_procedimiento  p  ON p.receta_id = r.id
LEFT JOIN puntos_criticos     pc ON pc.receta_id = r.id
LEFT JOIN puntos_clave        pk ON pk.receta_id = r.id
LEFT JOIN errores_frecuentes  ef ON ef.receta_id = r.id
LEFT JOIN criterios_evaluacion ce ON ce.receta_id = r.id
WHERE r.id = $1
GROUP BY r.id;
```

Esta consulta reconstruye el objeto completo de la receta en un solo roundtrip a la base de datos, devolviendo los arrays como JSON agregado — manteniendo la misma interfaz de datos que esperan los componentes Angular, minimizando cambios en la capa de presentación.